

The background of the entire image is a light gray grid. Scattered across this grid are various patterns of green squares, resembling pixel art. These patterns are located in the corners and around the central text, creating a decorative border. Some patterns are small clusters, while others are more complex shapes.

# GRA W ŻYCIU

**Implementacja symulacji komórkowej w  
technologii JavaFX**

# GŁÓWNE ZAŁOŻENIA PROJEKTU

- **Nieskończona plansza** – Brak sztywnych granic tablicy.
- **JavaFX** – Renderowanie sprzętowe (Canvas), nowoczesne UI.
- **Edytor zasad** – Możliwość definiowania własnych reguł przeżywania/umierania komórek.
- **Kamera** – Zoomowanie, przesuwanie widoku z użyciem myszy.
- **Modyfikacja planszy** – Dodawanie i usuwanie komórek z użyciem przycisków myszy
- **Modyfikacja tempa symulacji**

# STRUKTURA DANYCH

**Problem:** Jak przechowywać nieskończoną planszę w skończonej pamięci RAM?

**Wybrane rozwiązanie:** HashSet<Point> przechowujących tylko żywe komórki

- **Klasa Point** zaimplementowana jako record (automatyczne hashCode, i equals i gettery).
- **Złożoność pamięciowa:**  $O(n)$ , gdzie  $n$  to liczba żywych komórek.

# GENEROWANIE POKOLEŃ

Algorytm "Mapy Częstości":

## 1. Faza analizy:

- Iterujemy wyłącznie po żywych komórkach.
- Każda żywa komórka zwiększa licznik żywych sąsiadów u każdego jej sąsiada.
- Tworzymy mapę: `Map<Point, Integer>` (Współrzędna, Liczba żywych sąsiadów).

## 2. Faza decyzji:

- Iterujemy po kluczach mapy (tylko komórki aktywne/z żywymi sąsiadami).
- Aplikujemy zasady (np. jeśli licznik == 3 → ożywa).

**3. Zaleta:** Brak konieczności sprawdzania pustych obszarów planszy. Szybkość działania niezależna od rozmiaru widocznego obszaru.

# PLIKI WEJŚCIOWE/WYJŚCIOWE

**Format zapisu stanu gry: JSON** Pozwala zapisać nie tylko układ komórek, ale też stan kamery i reguły.

```
{  
  "rules": { "born": [3], "survive": [2, 3] },  
  "camera": { "x": 150.0, "y": -40.5, "zoom": 1.5 },  
  "cells": [  
    { "x": 0, "y": 1 },  
    { "x": 1, "y": 2 },  
    { "x": 2, "y": 0 }  
  ]  
}
```



# STRUKTURA KLAS

## Logika:

- record Point(int x, int y) – Współrzędne.
- class GameBoard – Przechowuje HashSet<Point>.
- class SimulationEngine – Realizuje algorytm.

## View (Widok):

- class GameCanvas – Rysowanie siatki i komórek.
- class Camera – Przeliczanie współrzędnych (Świat ↔ Ekran).

## Controller (Sterowanie):

- class MainController – Obsługa zdarzeń myszy, timera animacji.

# TESTY PROGRAMU

Testy automatyczne (JUnit 5) skupione na logice i matematyce.

## 1. Testy Logiki Gry:

- Sprawdzenie zasad standardowych i zmodyfikowanych.
- Działanie struktur na ujemnych współrzędnych (np.  $(-1000, -1000)$ ).

## 2. Testy Matematyki (Kamera):

- Konwersja kliknięcia myszką na współrzędne planszy przy różnym Zoomie i przesunięciu.

## 3. Testy Integracyjne:

- Zapis i odczyt stanu do JSON.

The background features a light gray grid with various green pixel art patterns scattered across it. These patterns include small clusters of squares, some forming larger shapes like a cross or a small 'X', and others being more abstract arrangements of colored pixels.

# GRA W ŻYCIU

**Implementacja symulacji komórkowej w  
technologii JavaFX**